

第 8 讲：价值函数方法

第 8 讲：价值函数方法

在前几章中，状态价值和动作价值均以表格形式表示。表格方法直观易懂，但在处理大规模状态空间或动作空间时效率低下。为解决这一问题，本章引入价值函数方法，使用函数来近似价值，这已成为表示价值的标准方法。正是在这里，神经网络作为函数近似器被融入强化学习。价值函数的思想也可以扩展到策略函数，详见第 9 章。

本章与前后章节的关联如下：第 7 讲介绍了表格形式的 TD 学习算法，本章将 TD 学习推广到函数近似的情形；第 9 讲将函数近似的思想进一步推广到策略函数的表示。

8.1 价值表示：从表格到函数

我们通过一个例子来说明表格方法和函数近似方法的区别。

假设有 n 个状态 $\{s_i\}_{i=1}^n$ ，其状态价值为 $\{v_\pi(s_i)\}_{i=1}^n$ ，其中 π 是给定的策略。设 $\{\hat{v}(s_i)\}_{i=1}^n$ 为真实状态价值的估计。如果使用表格方法，估计值可以保存在下表所示的表格中，该表可以以数组或向量的形式存储在内存中。

状态	s_1	s_2	$\cdots$	s_n
估计值	$\hat{v}(s_1)$	$\hat{v}(s_2)$	$\cdots$	$\hat{v}(s_n)$

要检索或更新某个价值，只需直接读取或重写表格中的对应条目即可。

然而，上述表格中的数值也可以用函数来近似。具体而言，点集 $\{(s_i, \hat{v}(s_i))\}_{i=1}^n$ 可以看作 n 个数据点，用一条曲线来拟合。最简单的曲线是直线，可以描述为：

$$\hat{v}(s; \mathbf{w}) = as + b = \underbrace{\begin{bmatrix} s & 1 \end{bmatrix}}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \end{bmatrix}}_{\mathbf{w}} = \phi^T(s) \mathbf{w} \quad (8.1)$$

其中 $\hat{v}(s; \mathbf{w})$ 是用于近似 $v_\pi(s)$ 的函数，由状态 s 和参数向量 $\mathbf{w} \in \mathbb{R}^2$ 共同决定。 $\hat{v}(s; \mathbf{w})$ 有时也写作 $\hat{v}_{\mathbf{w}}(s)$ 。这里 $\phi(s) \in \mathbb{R}^2$ 称为状态 s 的特征向量。

表格方法与函数近似方法在价值的检索和更新方面存在显著差异。

- **如何检索价值：**当价值以表格形式表示时，可以直接读取表中对应的条目。而当价值以函数形式表示时，需要将状态 s 输入函数并计算函数值。对于 (8.1) 中的例子，我们需要先计算特征向量 $\phi(s)$ ，然后计算 $\phi^T(s) \mathbf{w}$ 。如果函数是人工神经网络，则需要从输入层到输出层的前向传播。函数近似方法在存储方面更加高效。具体而言，表格方法需要存储 n 个值，而函数近似方法只需要存储一个低维参数向量 $\mathbf{w}$ ，从而极大提升存储效率。但这种好处是有代价的：状态价值可能无法被函数精确表示。从根本上看，用低维向量表示高维数据集必然会损失一些信息。因此，函数近似方法以牺牲精度为代价换取存储效率。
- **如何更新价值：**当价值以表格形式表示时，可以直接重写表中对应的条目。而当价值以函数形式表示时，更新某个价值的方法完全不同：必须通过更新 $\mathbf{w}$ 来间接改变价值。如何通过更新 $\mathbf{w}$ 来寻找最优状态价值将在后文详细讨论。

由于状态价值的更新方式不同，函数近似方法还具有另一个优点：其泛化能力比表格方法更强。使用表格方法时，只有在某个状态被访问到时才能更新该状态的价值，未访问状态的价值无法更新。而使用函数近似方法时，需要通过更新 $\mathbf{w}$ 来更新某个状态的价值， $\mathbf{w}$ 的更新也会影响其他状态的价值，即便这些状态尚未被访问。因此，一个状态的经验样本可以泛化并帮助估计其他状态的价值。

以上分析可以通过一个包含三个状态 $\{s_1, s_2, s_3\}$ 的例子来说明。假设我们有状态 s_3 的经验样本，并希望更新 $\hat{v}(s_3)$ 。使用表格方法时，只能更新 $\hat{v}(s_3)$ ，而不会改变 $\hat{v}(s_1)$ 或 $\hat{v}(s_2)$ 。使用函数近似方法时，更新 $\mathbf{w}$ 不仅可以更新 $\hat{v}(s_3)$ ，同时也会改变 $\hat{v}(s_1)$ 和 $\hat{v}(s_2)$ 。因此， s_3 的经验样本可以帮助更新其邻近状态的价值。

除了直线，我们还可以使用具有更强近似能力的复杂函数。例如，考虑一个二阶多项式：

$$\hat{v}(s; \mathbf{w}) = as^2 + bs + c = \underbrace{[s^2, s, 1]}_{\phi^T(s)} \underbrace{\begin{bmatrix} a \\ b \\ c \end{bmatrix}}_{\mathbf{w}} = \phi^T(s) \mathbf{w} \quad (8.2)$$

甚至可以使用更高阶的多项式曲线来拟合数据点。随着曲线阶数的增加，近似精度会提高，但参数向量的维数也会增大，需要更多的存储和计算资源。

注意，(8.1) 或 (8.2) 中的 $\hat{v}(s; \mathbf{w})$ 关于 $\mathbf{w}$ 是线性的（尽管关于 s 可能非线性）。这类方法称为**线性函数近似**，是最简单的函数近似方法。要实现线性函数近似，需要选择合适的特征向量 $\phi(s)$ 。特征向量的选择需要一定的先验知识：对任务理解越深，越能选择好的特征向量。例如，如果事先知道数据点大致分布在一条直线上，就可以用直线来拟合。但实践中通常不具备这样的先验知识，因此常用的方法是使用人工神经网络作为非线性函数近似器。

另一个重要问题是如何求最优参数向量。如果已知 $\{v_\pi(s_i)\}_{i=1}^n$ ，这就变成了一个最小二乘问题，可以通过优化以下目标函数来获得最优参数：

$$J = \frac{1}{n} \sum_{i=1}^n (\hat{v}(s_i; \mathbf{w}) - v_\pi(s_i))^2 = \frac{1}{n} \sum_{i=1}^n (\phi^T(s_i) \mathbf{w} - v_\pi(s_i))^2 = \frac{1}{n} \left\| \underbrace{\begin{bmatrix} \phi^T(s_1) \\ \vdots \\ \phi^T(s_n) \end{bmatrix}}_{\Phi} \mathbf{w} - \underbrace{\begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}}_{\mathbf{v}_\pi} \right\|^2 := \frac{1}{n} \|\Phi \mathbf{w} - \mathbf{v}_\pi\|^2$$

其中

$$\Phi := \begin{bmatrix} \phi^T(s_1) \\ \vdots \\ \phi^T(s_n) \end{bmatrix} \in \mathbb{R}^{n \times 2}, \quad \mathbf{v}_\pi := \begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix} \in \mathbb{R}^n$$

可以证明，该最小二乘问题的最优解为：

$$\mathbf{w}^* = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{v}_\pi$$

本节介绍的曲线拟合示例说明了价值函数近似的基本思想，下一节将正式引入这一思想。

8.2 基于函数近似的状态价值 TD 学习

本节介绍如何将函数近似方法融入 TD 学习，以估计给定策略的状态价值。该算法将在 8.3 节中扩展为学习动作价值和最优策略。

本节内容丰富且连贯性强，建议先概览以下内容再深入细节：

- 函数近似方法被形式化为一个优化问题，其目标函数在 8.2.1 节中介绍。用于优化该目标函数的 TD 学习算法在 8.2.2 节中介绍。
- 要应用 TD 学习算法，需要选择合适的特征向量，8.2.3 节讨论这一问题。
- 8.2.4 节给出示例，演示 TD 算法以及不同特征向量的影响。
- 8.2.5 节对 TD 算法进行理论分析，这部分数学难度较大，读者可根据兴趣选择性阅读。

8.2.1 目标函数

设 $v_\pi(s)$ 和 $\hat{v}(s; \mathbf{w})$ 分别为 $s \in \mathcal{S}$ 的真实状态价值和近似状态价值。目标是寻找最优的 $\mathbf{w}$ ，使得 $\hat{v}(s; \mathbf{w})$ 对每个 s 都能最佳地近似 $v_\pi(s)$ 。具体的目标函数为：

$$J(\mathbf{w}) = \mathbb{E}[(v_\pi(S) - \hat{v}(S; \mathbf{w}))^2] \quad (1)$$

其中期望是关于随机变量 $S \in \mathcal{S}$ 计算的。 S 是随机变量，那么其概率分布是什么？这个问题对于理解目标函数至关重要。定义 S 的概率分布有以下几种方式。

- **均匀分布**：将所有状态视为同等重要，将每个状态的概率设为 $1/n$ 。此时目标函数变为

$$J(\mathbf{w}) = \frac{1}{n} \sum_{s \in \mathcal{S}} (v_\pi(s) - \hat{v}(s; \mathbf{w}))^2$$

这是所有状态近似误差的平均值。然而，这种方式没有考虑给定策略下马尔可夫过程的真实动态特性。某些状态可能很少被某个策略访问，因此将所有状态同等看待可能并不合理。

- **平稳分布**（本章的重点）：平稳分布描述了马尔可夫决策过程的长期行为。具体而言，智能体在充分长的时间内执行某个给定策略后，其位于任一状态的概率可以由平稳分布描述（详见专栏 8.1）。设 $\{d_\pi(s)\}_{s \in \mathcal{S}}$ 为策略 π 下马尔可夫过程的平稳分布，即长期运行后智能体访问状态 s 的概率为 $d_\pi(s)$ 。由定义可知 $\sum_{s \in \mathcal{S}} d_\pi(s) = 1$ 。则目标函数可重写为：

$$J(\mathbf{w}) = \sum_{s \in \mathcal{S}} d_\pi(s) (v_\pi(s) - \hat{v}(s; \mathbf{w}))^2 \quad (2)$$

这是近似误差的加权平均，访问概率更高的状态被赋予更大的权重。

值得注意的是， $d_\pi(s)$ 的具体值并非平凡可求，因为需要知道状态转移概率矩阵 $\mathbf{P}_\pi$ （见专栏 8.1）。但幸运的是，如下一节所示，我们并不需要计算 $d_\pi(s)$ 的具体值来最小化该目标函数。此外，(8.4) 和 (8.5) 是在状态数量有限的前提下引入的，当状态空间连续时，可将求和替换为积分。

专栏 8.1: 马尔可夫决策过程的平稳分布

分析平稳分布的关键工具是策略 π 下的概率转移矩阵 $\mathbf{P}_\pi \in \mathbb{R}^{n \times n}$ 。若状态编号为 $s_1, \dots, s_n$ ，则 $[\mathbf{P}_\pi]_{ij}$ 定义为智能体从 s_i 转移到 s_j 的概率。

$\mathbf{P}_\pi^k$ 的含义 ($k = 1, 2, 3, \dots$)。首先需要考察 $\mathbf{P}_\pi^k$ 中各条目的含义。智能体从 s_i 转移到 s_j 恰好使用 k 步的概率记为

$$p_{ij}^{(k)} = \Pr(S_{t_k} = j \mid S_{t_0} = i)$$

由 $\mathbf{P}_\pi$ 的定义可知 $[\mathbf{P}_\pi]_{ij} = p_{ij}^{(1)}$ ，即 $[\mathbf{P}_\pi]_{ij}$ 是恰好用一步从 s_i 转移到 s_j 的概率。其次，考虑 $\mathbf{P}_\pi^2$ ：

$$[\mathbf{P}_\pi^2]_{ij} = [\mathbf{P}_\pi \mathbf{P}_\pi]_{ij} = \sum_{q=1}^n [\mathbf{P}_\pi]_{iq} [\mathbf{P}_\pi]_{qj}$$

因为 $[\mathbf{P}_\pi]_{iq} [\mathbf{P}_\pi]_{qj}$ 是从 s_i 转移到 s_q 再转移到 s_j 的联合概率，所以 $[\mathbf{P}_\pi^2]_{ij}$ 是恰好用两步从 s_i 转移到 s_j 的概率，即 $[\mathbf{P}_\pi^2]_{ij} = p_{ij}^{(2)}$ 。类似地，有 $[\mathbf{P}_\pi^k]_{ij} = p_{ij}^{(k)}$ 。

平稳分布的定义。设 $\mathbf{d}_0 \in \mathbb{R}^n$ 为初始时刻状态的分布向量, $\mathbf{d}_k \in \mathbb{R}^n$ 为从 $\mathbf{d}_0$ 出发恰好经过 k 步后的分布向量, 则有:

$$d_k(s_i) = \sum_{j=1}^n d_0(s_j) [\mathbf{P}_\pi^k]_{ji}, \quad i = 1, 2, \dots$$

其矩阵-向量形式为:

$$\mathbf{d}_k^T = \mathbf{d}_0^T \mathbf{P}_\pi^k$$

考虑马尔可夫过程的长期行为, 在特定条件下有:

$$\lim_{k \rightarrow \infty} \mathbf{P}_\pi^k = \mathbf{1}_n \mathbf{d}_\pi^T$$

其中 $\mathbf{1}_n = [1, \dots, 1]^T \in \mathbb{R}^n$, $\mathbf{1}_n \mathbf{d}_\pi^T$ 是一个所有行都等于 $\mathbf{d}_\pi^T$ 的常数矩阵。代入上式得:

$$\lim_{k \rightarrow \infty} \mathbf{d}_k^T = \mathbf{d}_0^T \lim_{k \rightarrow \infty} \mathbf{P}_\pi^k = \mathbf{d}_0^T \mathbf{1}_n \mathbf{d}_\pi^T = \mathbf{d}_\pi^T$$

这意味着状态分布 $\mathbf{d}_k$ 收敛到一个常值 $\mathbf{d}_\pi$, 称为**极限分布**。极限分布取决于系统模型和策略 π , 但与初始分布 $\mathbf{d}_0$ 无关。 $\mathbf{d}_\pi$ 的值可通过下式计算: 对 $\mathbf{d}_k^T = \mathbf{d}_{k-1}^T \mathbf{P}_\pi$ 两边取极限, 得 $\mathbf{d}_\pi^T = \mathbf{d}_\pi^T \mathbf{P}_\pi$ 。因此, $\mathbf{d}_\pi$ 是 $\mathbf{P}_\pi$ 对应于特征值 1 的左特征向量, 称为**平稳分布**。有 $\sum_{s \in \mathcal{S}} d_\pi(s) = 1$ 且 $d_\pi(s) > 0$ 对所有 $s \in \mathcal{S}$ 成立。

平稳分布唯一性的条件。满足唯一平稳 (或极限) 分布的一类马尔可夫过程是**不可约 (或正则)**马尔可夫过程。相关定义如下:

- 若存在有限整数 k 使得 $[\mathbf{P}_\pi^k]_{ij} > 0$, 则称 s_j 从 s_i 可达。
- 若 s_i 和 s_j 互相可达, 则称这两个状态**互通**。
- 若所有状态都互通, 则马尔可夫过程称为**不可约的**。
- 若存在 $k \geq 1$ 使得 $[\mathbf{P}_\pi^k]_{ij} > 0$ 对所有 i, j 成立, 则马尔可夫过程称为**正则的**。此时有 $\mathbf{P}_\pi^k > 0$ (按元素)。正则马尔可夫过程也是不可约的, 反之不一定成立。但如果不可约且存在 i 使得 $[\mathbf{P}_\pi]_{ii} > 0$, 则也是正则的。此外, 若 $\mathbf{P}_\pi^k > 0$, 则对任意 $k' \geq k$ 都有 $\mathbf{P}_\pi^{k'} > 0$ 。由此可知 $d_\pi(s) > 0$ 对每个 s 成立。

能产生唯一平稳分布的策略。一旦策略给定, MDP 就变为马尔可夫过程, 其长期行为由策略和系统模型共同决定。什么类型的策略能产生正则马尔可夫过程? 一般来说, 探索性策略 (如 ε -greedy 策略) 可做到这一点, 因为探索性策略在任何状态下选择任何动作的概率均为正, 因此只要系统模型允许, 状态之间就可以互通。

8.2.2 优化算法

为最小化目标函数 $J(\mathbf{w})$, 可以使用梯度下降算法:

$$\mathbf{w}_{k+1} = \mathbf{w}_k - \alpha_k \nabla_{\mathbf{w}} J(\mathbf{w}_k)$$

其中

$$\nabla_{\mathbf{w}} J(\mathbf{w}_k) = \nabla_{\mathbf{w}} \mathbb{E}[(v_\pi(S) - \hat{v}(S; \mathbf{w}_k))^2] = -2\mathbb{E}[(v_\pi(S) - \hat{v}(S; \mathbf{w}_k)) \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w}_k)]$$

因此梯度下降算法为:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + 2\alpha_k \mathbb{E}[(v_\pi(S) - \hat{v}(S; \mathbf{w}_k)) \nabla_{\mathbf{w}} \hat{v}(S; \mathbf{w}_k)] \quad (3)$$

其中系数 2 可以被合并到 α_k 中。该算法需要计算期望, 按照随机梯度下降的思想, 用随机梯度替换真实梯度, 得到:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (v_\pi(s_t) - \hat{v}(s_t; \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(s_t; \mathbf{w}_t) \quad (4)$$

其中 s_t 是 S 在时刻 t 的样本。

值得注意的是, (8.12) 并非可直接实现的算法, 因为它需要真实的状态价值 v_π , 而 v_π 是未知的, 必须通过近似来估计。可以用以下两种方法替代 $v_\pi(s_t)$:

- **蒙特卡洛方法:** 假设有一个轨迹 $(s_0, r_1, s_1, r_2, \dots)$, 令 G_t 为从 s_t 开始的折扣回报, 则 G_t 可以作为 $v_\pi(s_t)$ 的近似。算法变为:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (G_t - \hat{v}(s_t; \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(s_t; \mathbf{w}_t)$$

这就是带函数近似的蒙特卡洛学习算法。

- **时间差分方法:** 按照 TD 学习的思想, $r_{t+1} + \gamma \hat{v}(s_{t+1}; \mathbf{w}_t)$ 可以作为 $v_\pi(s_t)$ 的近似。算法变为:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t [r_{t+1} + \gamma \hat{v}(s_{t+1}; \mathbf{w}_t) - \hat{v}(s_t; \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{v}(s_t; \mathbf{w}_t) \quad (5)$$

这就是带函数近似的 TD 学习算法, 总结在算法 8.1 中。

理解 (8.13) 中的 TD 算法对学习本章其他算法至关重要。需要注意的是, (8.13) 只能学习给定策略的状态价值, 8.3.1 和 8.3.2 节将把它扩展为能够学习动作价值的算法。

8.2.3 函数近似器的选择

要应用 (8.13) 中的 TD 算法, 需要选择合适的 $\hat{v}(s; \mathbf{w})$ 。有两种方式: 第一种是使用人工神经网络作为非线性函数近似器, 网络的输入是状态, 输出是 $\hat{v}(s; \mathbf{w})$, 网络的参数是 $\mathbf{w}$ 。第二种是简单地使用线性函数:

$$\hat{v}(s; \mathbf{w}) = \phi^T(s) \mathbf{w}$$

其中 $\phi(s) \in \mathbb{R}^m$ 是状态 s 的特征向量。 $\phi(s)$ 和 $\mathbf{w}$ 的长度都等于 m , 通常远小于状态数。在线性情况下, 梯度为 $\nabla_{\mathbf{w}} \hat{v}(s; \mathbf{w}) = \phi(s)$, 代入 (8.13) 得:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t [r_{t+1} + \gamma \phi^T(s_{t+1}) \mathbf{w}_t - \phi^T(s_t) \mathbf{w}_t] \phi(s_t) \quad (6)$$

这就是带线性函数近似的 TD 学习算法, 简称为 **TD-Linear**。

线性情形的理论性质远好于非线性情形, 但其近似能力有限。对于复杂任务, 选择合适的特征向量也并非易事。相比之下, 人工神经网络可以作为黑盒通用非线性近似器来近似价值, 使用更加友好。

不过, 研究线性情形仍然很有意义。更好地理解线性情形有助于更好地掌握函数近似方法的思想。而且, 线性情形对于本书中考虑的简单网格世界任务已经足够。更重要的是, 线性情形仍然很强大, 因为表格方法可以视为一种特殊的线性情形, 详见专栏 8.2。

证明

专栏 8.2: 表格 TD 学习是 TD-Linear 的特殊情形

下面证明第 7 章中的表格 TD 算法 (7.1) 是 TD-Linear 算法 (8.14) 的特殊情形。

考虑以下特殊特征向量, 对任意 $s \in \mathcal{S}$:

$$\phi(s) = \mathbf{e}_s \in \mathbb{R}^n$$

其中 $\mathbf{e}_s$ 是对应于 s 的条目为 1、其余条目为 0 的向量。此时有:

$$\hat{v}(s; \mathbf{w}) = \mathbf{e}_s^T \mathbf{w} = w(s)$$

其中 $w(s)$ 是 $\mathbf{w}$ 中对应于 s 的条目。代入 (8.14) 得:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t)) \mathbf{e}_{s_t}$$

上式仅更新 $w_t(s_t)$ 这一条目。两边同时左乘 $e_{s_t}^T$ 得：

$$w_{t+1}(s_t) = w_t(s_t) + \alpha_t(r_{t+1} + \gamma w_t(s_{t+1}) - w_t(s_t))$$

这正是表格 TD 算法 (7.1)。

总之，通过选择特征向量 $\phi(s) = e_s$ ，TD-Linear 算法就变成了表格 TD 算法。

8.2.4 示例说明

下面通过一些示例演示如何使用 TD-Linear 算法 (8.14) 来估计给定策略的状态价值，并展示如何选择特征向量。

考虑一个 5×5 的网格世界。给定策略在每个状态下以 0.2 的概率选择任一动作。目标是估计该策略下的状态价值，共有 25 个状态价值。仿真设置如下：由给定策略生成 500 个 episode，每个 episode 有 500 步，从一个随机选择的状态-动作对按均匀分布出发。此外，每次仿真试验中参数向量 $\mathbf{w}$ 随机初始化，每个元素从均值为 0、标准差为 1 的标准正态分布中抽取。设 $r_{\text{forbidden}} = r_{\text{boundary}} = -1$ ， $r_{\text{target}} = 1$ ， $\gamma = 0.9$ 。

要使用 TD-Linear 算法，首先需要选择特征向量 $\phi(s)$ 。以下给出一些不同的选择方式。

第一类特征向量基于多项式。在网格世界中，状态 s 对应一个二维位置。令 x 和 y 分别表示 s 的列索引和行索引。为避免数值问题，将 x 和 y 归一化到 $[-1, +1]$ 区间。最简单的特征向量是：

$$\phi(s) = \begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$$

此时 $\hat{v}(s; \mathbf{w}) = \phi^T(s) \mathbf{w} = w_1 x + w_2 y$ ，表示一个过原点的二维平面。由于状态价值的曲面可能不过原点，需要引入偏置项。为此考虑如下三维特征向量：

$$\phi(s) = \begin{bmatrix} 1 \\ x \\ y \end{bmatrix} \in \mathbb{R}^3 \quad (7)$$

此时 $\hat{v}(s; \mathbf{w}) = w_1 + w_2 x + w_3 y$ ，对应于一个可以不过原点的平面。为增强近似能力，可以增加特征向量的维数：

$$\phi(s) = [1, x, y, x^2, y^2, xy]^T \in \mathbb{R}^6 \quad (8)$$

$$\phi(s) = [1, x, y, x^2, y^2, xy, x^3, y^3, x^2 y, xy^2]^T \in \mathbb{R}^{10} \quad (9)$$

特征向量越长，状态价值近似越精确。但由于这些线性近似器的近似能力有限，在所有三种情况下，估计误差都无法收敛到零。

此外还有基于傅里叶基和瓦片编码的特征向量。将每个状态的 x 和 y 归一化到 $[0, 1]$ 区间。得到的傅里叶特征向量为：

$$\phi(s) = \begin{bmatrix} \vdots \\ \cos(\pi(c_1 x + c_2 y)) \\ \vdots \end{bmatrix} \in \mathbb{R}^{(q+1)^2}$$

其中 π 表示圆周率 (3.1415...)，不是策略。 c_1 或 c_2 可以取 $\{0, 1, \dots, q\}$ 中的任意整数， q 是用户指定的整数。因此 (c_1, c_2) 共有 $(q+1)^2$ 种可能的取值， $\phi(s)$ 的维数为 $(q+1)^2$ 。例如 $q = 1$ 时：

$$\phi(s) = \begin{bmatrix} \cos(\pi(0x + 0y)) \\ \cos(\pi(0x + 1y)) \\ \cos(\pi(1x + 0y)) \\ \cos(\pi(1x + 1y)) \end{bmatrix} = \begin{bmatrix} 1 \\ \cos(\pi y) \\ \cos(\pi x) \\ \cos(\pi(x + y)) \end{bmatrix} \in \mathbb{R}^4$$

当 $q = 1, 2, 3$ 时, 特征向量的维数分别为 4, 9, 16。特征向量维数越高, 状态价值近似越精确。

8.2.5 理论分析

至此, 带函数近似的 TD 学习的故事已经讲完。这个故事从目标函数 (8.3) 出发, 为优化该目标函数引入了随机算法 (8.12), 最后将算法中未知的真实价值函数替换为近似, 得到了 TD 算法 (8.13)。尽管这个故事有助于理解价值函数近似的基本思想, 但它并不严格。例如, (8.13) 中的算法实际上并不最小化 (8.3) 中的目标函数。

下面给出 (8.13) 中 TD 算法的理论分析, 以揭示该算法为何有效以及它解决了什么数学问题。由于一般非线性近似器难以分析, 本部分只考虑线性情形。这部分数学难度较大, 读者可根据兴趣选择性阅读。

收敛性分析

为研究 (8.13) 的收敛性质, 首先考虑以下确定性算法:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \mathbb{E} \left[(r_{t+1} + \gamma \boldsymbol{\phi}^T(s_{t+1}) \mathbf{w}_t - \boldsymbol{\phi}^T(s_t) \mathbf{w}_t) \boldsymbol{\phi}(s_t) \right] \quad (10)$$

其中期望是对随机变量 s_t, s_{t+1}, r_{t+1} 计算的。 s_t 的分布假设为平稳分布 d_π 。(8.19) 是确定性的, 因为计算期望后随机变量 s_t, s_{t+1}, r_{t+1} 都消失了。

为什么要考虑这个确定性算法? 首先, 该确定性算法的收敛性更容易分析。其次, 更重要的是, 该确定性算法的收敛蕴含了随机 TD 算法 (8.13) 的收敛, 因为 (8.13) 可以看作 (8.19) 的随机梯度下降实现。因此只需研究确定性算法的收敛性质。

虽然 (8.19) 的表达式初看可能复杂, 但可以通过以下定义简化。令

$$\boldsymbol{\Phi} = \begin{bmatrix} \vdots \\ \boldsymbol{\phi}^T(s) \\ \vdots \end{bmatrix} \in \mathbb{R}^{n \times m}, \quad \mathbf{D} = \begin{bmatrix} \ddots & & \\ & d_\pi(s) & \\ & & \ddots \end{bmatrix} \in \mathbb{R}^{n \times n} \quad (11)$$

其中 $\boldsymbol{\Phi}$ 是包含所有特征向量的矩阵, $\mathbf{D}$ 是对角线为平稳分布的对角矩阵。

定理 8.1

(8.19) 中的期望可以重写为:

$$\mathbb{E} \left[(r_{t+1} + \gamma \boldsymbol{\phi}^T(s_{t+1}) \mathbf{w}_t - \boldsymbol{\phi}^T(s_t) \mathbf{w}_t) \boldsymbol{\phi}(s_t) \right] = \mathbf{b} - \mathbf{A} \mathbf{w}_t$$

其中

$$\mathbf{A} := \boldsymbol{\Phi}^T \mathbf{D} (\mathbf{I} - \gamma \mathbf{P}_\pi) \boldsymbol{\Phi} \in \mathbb{R}^{m \times m} \quad (12)$$

$$\mathbf{b} := \boldsymbol{\Phi}^T \mathbf{D} \mathbf{r}_\pi \in \mathbb{R}^m \quad (13)$$

这里 $\mathbf{P}_\pi, \mathbf{r}_\pi$ 是贝尔曼方程 $\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_\pi$ 中的两项, $\mathbf{I}$ 是适当维数的单位矩阵。

借助引理 8.1 的表达式, 确定性算法 (8.19) 可重写为:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (\mathbf{b} - \mathbf{A} \mathbf{w}_t) \quad (14)$$

这是一个简单的确定性过程。

首先, $\mathbf{w}_t$ 的收敛值是什么? 假设 $\mathbf{w}_t$ 收敛到一个常值 $\mathbf{w}^*$ (当 $t \rightarrow \infty$), 则 (8.22) 蕴含 $\mathbf{w}^* = \mathbf{w}^* + \alpha_\infty(\mathbf{b} - \mathbf{A}\mathbf{w}^*)$, 由此得 $\mathbf{b} - \mathbf{A}\mathbf{w}^* = 0$, 即:

$$\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$$

关于该收敛值有以下几点说明。

- $\mathbf{A}$ 是否可逆? 答案是可以。事实上 $\mathbf{A}$ 不仅是可逆的, 还是正定的。正定性的证明见专栏 8.4。
- $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$ 的解释是什么? 它实际上是最小化投影贝尔曼误差的最优解, 详见 8.2.5 节。
- 表格方法是特殊情况。当 $\mathbf{w}$ 的维数等于 $n = |\mathcal{S}|$ 且 $\phi(s) = [0, \dots, 1, \dots, 0]^T$ (对应 s 的条目为 1) 时, 有 $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b} = \mathbf{v}_\pi$ 。这表明要学习的参数向量实际上就是真实的状态价值。这与表格 TD 算法是 TD-Linear 算法的特例(专栏 8.2)一致。验证如下: 此时 $\Phi = \mathbf{I}$, 因此 $\mathbf{A} = \Phi^T D(\mathbf{I} - \gamma \mathbf{P}_\pi) \Phi = D(\mathbf{I} - \gamma \mathbf{P}_\pi)$, $\mathbf{b} = \Phi^T D \mathbf{r}_\pi = D \mathbf{r}_\pi$ 。于是 $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b} = (\mathbf{I} - \gamma \mathbf{P}_\pi)^{-1} D^{-1} D \mathbf{r}_\pi = (\mathbf{I} - \gamma \mathbf{P}_\pi)^{-1} \mathbf{r}_\pi = \mathbf{v}_\pi$ 。

其次, 可以证明 (8.22) 中的 $\mathbf{w}_t$ 收敛到 $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$ (当 $t \rightarrow \infty$)。证明方式如下:

证明一: 定义收敛误差 $\tilde{\mathbf{w}}_t := \mathbf{w}_t - \mathbf{w}^*$ 。只需证明 $\tilde{\mathbf{w}}_t$ 收敛到零。将 $\mathbf{w}_t = \tilde{\mathbf{w}}_t + \mathbf{w}^*$ 代入 (8.22) 得:

$$\tilde{\mathbf{w}}_{t+1} = \tilde{\mathbf{w}}_t - \alpha_t \mathbf{A} \tilde{\mathbf{w}}_t = (\mathbf{I} - \alpha_t \mathbf{A}) \tilde{\mathbf{w}}_t$$

因此 $\tilde{\mathbf{w}}_{t+1} = (\mathbf{I} - \alpha_t \mathbf{A}) \cdots (\mathbf{I} - \alpha_0 \mathbf{A}) \tilde{\mathbf{w}}_0$ 。考虑简单情形 $\alpha_t = \alpha$, 则有:

$$\|\tilde{\mathbf{w}}_{t+1}\|_2 \leq \|\mathbf{I} - \alpha \mathbf{A}\|_2^{t+1} \|\tilde{\mathbf{w}}_0\|_2$$

当 $\alpha > 0$ 足够小时, $\|\mathbf{I} - \alpha \mathbf{A}\|_2 < 1$ 成立 (因为 $\mathbf{A}$ 正定), 因此 $\tilde{\mathbf{w}}_t \rightarrow 0$ 。

证明二: 考虑 $g(\mathbf{w}) := \mathbf{b} - \mathbf{A}\mathbf{w}$ 。由于 $\mathbf{w}^*$ 是 $g(\mathbf{w}) = 0$ 的根, 这实际上是一个求根问题。(8.22) 中的算法实际上是 Robbins-Monro (RM) 算法, 当 $\sum_t \alpha_t = \infty$ 且 $\sum_t \alpha_t^2 < \infty$ 时 $\mathbf{w}_t$ 收敛到 $\mathbf{w}^*$ 。

证明

专栏 8.3: 引理 8.1 的证明

利用全期望公式:

$$\begin{aligned} & \mathbb{E}[r_{t+1}\phi(s_t) + \phi(s_t)(\gamma\phi^T(s_{t+1}) - \phi^T(s_t))\mathbf{w}_t] \\ &= \sum_{s \in \mathcal{S}} d_\pi(s) \mathbb{E}[r_{t+1}\phi(s_t) + \phi(s_t)(\gamma\phi^T(s_{t+1}) - \phi^T(s_t))\mathbf{w}_t \mid s_t = s] \\ &= \sum_{s \in \mathcal{S}} d_\pi(s) \mathbb{E}[r_{t+1}\phi(s_t) \mid s_t = s] + \sum_{s \in \mathcal{S}} d_\pi(s) \mathbb{E}[\phi(s_t)(\gamma\phi^T(s_{t+1}) - \phi^T(s_t))\mathbf{w}_t \mid s_t = s] \end{aligned}$$

首先, 第一项注意到 $\mathbb{E}[r_{t+1}\phi(s_t) \mid s_t = s] = \phi(s)\mathbb{E}[r_{t+1} \mid s_t = s] = \phi(s)r_\pi(s)$, 其中 $r_\pi(s) = \sum_a \pi(a|s) \sum_r r p(r|s, a)$ 。因此第一项可重写为 $\sum_{s \in \mathcal{S}} d_\pi(s) \phi(s) r_\pi(s) = \Phi^T D \mathbf{r}_\pi$ 。

其次, 第二项中:

$$\begin{aligned} & \mathbb{E}[\phi(s_t)(\gamma\phi^T(s_{t+1}) - \phi^T(s_t))\mathbf{w}_t \mid s_t = s] \\ &= -\mathbb{E}[\phi(s_t)\phi^T(s_t)\mathbf{w}_t \mid s_t = s] + \mathbb{E}[\gamma\phi(s_t)\phi^T(s_{t+1})\mathbf{w}_t \mid s_t = s] \\ &= -\phi(s)\phi^T(s)\mathbf{w}_t + \gamma\phi(s)\mathbb{E}[\phi^T(s_{t+1}) \mid s_t = s]\mathbf{w}_t \\ &= -\phi(s)\phi^T(s)\mathbf{w}_t + \gamma\phi(s) \sum_{s' \in \mathcal{S}} p(s'|s)\phi^T(s')\mathbf{w}_t \end{aligned}$$

因此第二项变为:

$$\sum_{s \in \mathcal{S}} d_\pi(s) \phi(s) \left[-\phi(s) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s) \phi(s') \right]^T \mathbf{w}_t = \Phi^T D (-\Phi + \gamma \mathbf{P}_\pi \Phi) \mathbf{w}_t = -\Phi^T D (\mathbf{I} - \gamma \mathbf{P}_\pi) \Phi \mathbf{w}_t$$

合并两项即得：

$$\mathbb{E}\left[(r_{t+1} + \gamma \phi^T(s_{t+1})\mathbf{w}_t - \phi^T(s_t)\mathbf{w}_t)\phi(s_t)\right] = \Phi^T D \mathbf{r}_\pi - \Phi^T D(I - \gamma P_\pi) \Phi \mathbf{w}_t := \mathbf{b} - A \mathbf{w}_t$$

证明

专栏 8.4： 证明 $A = \Phi^T D(I - \gamma P_\pi) \Phi$ 可逆且正定

若对任意非零向量 $\mathbf{x}$ 有 $\mathbf{x}^T A \mathbf{x} > 0$ ，则 A 正定（记作 $A \succ 0$ ）。注意 A 可能不对称，但非对称矩阵也可以是正定的。

证明思路是：先证 $D(I - \gamma P_\pi) := M \succ 0$ ，则 $A = \Phi^T M \Phi \succ 0$ （因为 Φ 是满列秩的高型矩阵，假设特征向量线性无关）。

注意到 $M = \frac{M+M^T}{2} + \frac{M-M^T}{2}$ ，由于 $M - M^T$ 是反对称的， $\mathbf{x}^T(M - M^T)\mathbf{x} = 0$ 对任意 $\mathbf{x}$ 成立。因此 $M \succ 0$ 当且仅当 $M + M^T \succ 0$ 。为证明 $M + M^T \succ 0$ ，运用严格对角占优矩阵正定的结论。

首先， $(M + M^T)\mathbf{1}_n > 0$ 。因为 $M\mathbf{1}_n = D(I - \gamma P_\pi)\mathbf{1}_n = D(\mathbf{1}_n - \gamma \mathbf{1}_n) = (1 - \gamma)\mathbf{d}_\pi$ ，且 $M^T\mathbf{1}_n = (I - \gamma P_\pi^T)D\mathbf{1}_n = (I - \gamma P_\pi^T)\mathbf{d}_\pi = (1 - \gamma)\mathbf{d}_\pi$ （因为 $P_\pi^T \mathbf{d}_\pi = \mathbf{d}_\pi$ ）。因此 $(M + M^T)\mathbf{1}_n = 2(1 - \gamma)\mathbf{d}_\pi > 0$ （所有元素）。

其次，按元素的写法为 $\sum_{j=1}^n [M + M^T]_{ij} > 0$ ，即 $[M + M^T]_{ii} + \sum_{j \neq i} [M + M^T]_{ij} > 0$ 。根据 M 的表达式， M 的对角线元素为正、非对角线元素为非正，因此上述不等式可改写为：

$$|[M + M^T]_{ii}| > \sum_{j \neq i} |[M + M^T]_{ij}|$$

这表明 $M + M^T$ 严格对角占优，因此正定。证毕。

TD 学习最小化投影贝尔曼误差

上面已经证明 TD-Linear 算法收敛到 $\mathbf{w}^* = A^{-1}\mathbf{b}$ ，下面进一步说明 $\mathbf{w}^*$ 是使投影贝尔曼误差最小的最优解。为此，先回顾三种目标函数。

• 第一种目标函数：

$$J_E(\mathbf{w}) = \mathbb{E}[(v_\pi(S) - \hat{v}(S; \mathbf{w}))^2]$$

可用矩阵-向量形式重写为 $J_E(\mathbf{w}) = \|\hat{\mathbf{v}}(\mathbf{w}) - \mathbf{v}_\pi\|_D^2$ ，其中 $\|\mathbf{x}\|_D^2 = \mathbf{x}^T D \mathbf{x} = \|D^{1/2} \mathbf{x}\|_2^2$ 。这是讨论函数近似时能想到的最简单的目标函数，但依赖于未知的真实状态价值。

• 第二种目标函数是贝尔曼误差。因为 $\mathbf{v}_\pi$ 满足贝尔曼方程 $\mathbf{v}_\pi = \mathbf{r}_\pi + \gamma P_\pi \mathbf{v}_\pi$ ，期望估计值 $\hat{\mathbf{v}}(\mathbf{w})$ 也尽可能满足该方程。贝尔曼误差定义为：

$$J_{BE}(\mathbf{w}) = \|\hat{\mathbf{v}}(\mathbf{w}) - (\mathbf{r}_\pi + \gamma P_\pi \hat{\mathbf{v}}(\mathbf{w}))\|_D^2 := \|\hat{\mathbf{v}}(\mathbf{w}) - T_\pi(\hat{\mathbf{v}}(\mathbf{w}))\|_D^2$$

其中 $T_\pi(\cdot)$ 是贝尔曼算子，定义为 $T_\pi(\mathbf{x}) := \mathbf{r}_\pi + \gamma P_\pi \mathbf{x}$ 。最小化贝尔曼误差是标准的最小二乘问题。

• 第三种目标函数是投影贝尔曼误差。由于近似器的近似能力有限， $J_{BE}(\mathbf{w})$ 可能无法最小化到零。而可以被最小化到零的目标函数是投影贝尔曼误差：

$$J_{PBE}(\mathbf{w}) = \|\hat{\mathbf{v}}(\mathbf{w}) - M T_\pi(\hat{\mathbf{v}}(\mathbf{w}))\|_D^2$$

其中 $M \in \mathbb{R}^{n \times n}$ 是正交投影矩阵，几何上将任意向量投影到所有近似的空间上。

事实上，TD 学习算法 (8.13) 的目标是最小化投影贝尔曼误差 J_{PBE} ，而非 J_E 或 J_{BE} 。考虑线性情形 $\hat{\mathbf{v}}(\mathbf{w}) = \Phi \mathbf{w}$ ， Φ 的列空间是所有可能线性近似的集合。则 $M = \Phi(\Phi^T D \Phi)^{-1} \Phi^T D$ 是投影

矩阵。可以证明最小化 $J_{PBE}(\mathbf{w})$ 的解就是 $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$:

$$\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b} = \arg \min_{\mathbf{w}} J_{PBE}(\mathbf{w})$$

证明见专栏 8.5。

证明

专栏 8.5: 证明 $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$ 最小化 $J_{PBE}(\mathbf{w})$

由于 $J_{PBE}(\mathbf{w}) = 0$ 要求 $\hat{\mathbf{v}}(\mathbf{w}) - \mathbf{M}T_\pi(\hat{\mathbf{v}}(\mathbf{w})) = 0$, 只需研究 $\hat{\mathbf{v}}(\mathbf{w}) = \mathbf{M}T_\pi(\hat{\mathbf{v}}(\mathbf{w}))$ 的根。在线性情形下, 代入 $\hat{\mathbf{v}}(\mathbf{w}) = \Phi\mathbf{w}$ 及 $\mathbf{M}$ 的表达式:

$$\Phi\mathbf{w} = \Phi(\Phi^T D \Phi)^{-1} \Phi^T D(\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \Phi\mathbf{w})$$

由于 Φ 具有满列秩, $\Phi\mathbf{x} = \Phi\mathbf{y} \iff \mathbf{x} = \mathbf{y}$, 因此:

$$\begin{aligned} \mathbf{w} &= (\Phi^T D \Phi)^{-1} \Phi^T D(\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \Phi\mathbf{w}) \\ \iff \Phi^T D(\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \Phi\mathbf{w}) &= (\Phi^T D \Phi)\mathbf{w} \\ \iff \Phi^T D\mathbf{r}_\pi + \gamma \Phi^T D\mathbf{P}_\pi \Phi\mathbf{w} &= (\Phi^T D \Phi)\mathbf{w} \\ \iff \Phi^T D\mathbf{r}_\pi &= \Phi^T D(I - \gamma \mathbf{P}_\pi)\Phi\mathbf{w} \\ \iff \mathbf{w} &= (\Phi^T D(I - \gamma \mathbf{P}_\pi)\Phi)^{-1} \Phi^T D\mathbf{r}_\pi = \mathbf{A}^{-1}\mathbf{b} \end{aligned}$$

因此 $\mathbf{w}^* = \mathbf{A}^{-1}\mathbf{b}$ 是使 $J_{PBE}(\mathbf{w})$ 最小的最优解。

既然 TD 算法旨在最小化 J_{PBE} 而非 J_E , 自然要问估计价值 $\hat{\mathbf{v}}(\mathbf{w})$ 离真实状态价值 $\mathbf{v}_\pi$ 有多近。在线性情形下, 最小化投影贝尔曼误差的估计值为 $\hat{\mathbf{v}}(\mathbf{w}^*) = \Phi\mathbf{w}^*$, 其与真实状态价值 $\mathbf{v}_\pi$ 的偏差满足:

$$\|\hat{\mathbf{v}}(\mathbf{w}^*) - \mathbf{v}_\pi\|_D = \|\Phi\mathbf{w}^* - \mathbf{v}_\pi\|_D \leq \frac{1}{1-\gamma} \min_{\mathbf{w}} \|\hat{\mathbf{v}}(\mathbf{w}) - \mathbf{v}_\pi\|_D = \frac{1}{1-\gamma} \min_{\mathbf{w}} \sqrt{J_E(\mathbf{w})}$$

该不等式的证明见专栏 8.6。它表明 $\Phi\mathbf{w}^*$ 与 $\mathbf{v}_\pi$ 之间的差异被 $J_E(\mathbf{w})$ 的最小值所界定。然而这个界是宽松的, 尤其当 γ 接近 1 时, 因此主要具有理论价值。

证明

专栏 8.6: 误差界 (8.32) 的证明

注意到:

$$\begin{aligned} \|\Phi\mathbf{w}^* - \mathbf{v}_\pi\|_D &= \|\Phi\mathbf{w}^* - \mathbf{M}\mathbf{v}_\pi + \mathbf{M}\mathbf{v}_\pi - \mathbf{v}_\pi\|_D \\ &\leq \|\Phi\mathbf{w}^* - \mathbf{M}\mathbf{v}_\pi\|_D + \|\mathbf{M}\mathbf{v}_\pi - \mathbf{v}_\pi\|_D \\ &= \|\mathbf{M}T_\pi(\Phi\mathbf{w}^*) - \mathbf{M}T_\pi(\mathbf{v}_\pi)\|_D + \|\mathbf{M}\mathbf{v}_\pi - \mathbf{v}_\pi\|_D \end{aligned}$$

其中最后一步利用了 $\Phi\mathbf{w}^* = \mathbf{M}T_\pi(\Phi\mathbf{w}^*)$ 和 $\mathbf{v}_\pi = T_\pi(\mathbf{v}_\pi)$ 。代入:

$$\mathbf{M}T_\pi(\Phi\mathbf{w}^*) - \mathbf{M}T_\pi(\mathbf{v}_\pi) = \mathbf{M}(\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \Phi\mathbf{w}^*) - \mathbf{M}(\mathbf{r}_\pi + \gamma \mathbf{P}_\pi \mathbf{v}_\pi) = \gamma \mathbf{M}\mathbf{P}_\pi(\Phi\mathbf{w}^* - \mathbf{v}_\pi)$$

代入不等式得:

$$\|\Phi\mathbf{w}^* - \mathbf{v}_\pi\|_D \leq \gamma \|\mathbf{M}\|_D \|\mathbf{P}_\pi(\Phi\mathbf{w}^* - \mathbf{v}_\pi)\|_D + \|\mathbf{M}\mathbf{v}_\pi - \mathbf{v}_\pi\|_D$$

由于 $\|\mathbf{M}\|_D = 1$ 且 $\|\mathbf{P}_\pi \mathbf{x}\|_D \leq \|\mathbf{x}\|_D$ 对任意 $\mathbf{x}$ 成立 (证明见下文), 因此:

$$\|\Phi\mathbf{w}^* - \mathbf{v}_\pi\|_D \leq \gamma \|\Phi\mathbf{w}^* - \mathbf{v}_\pi\|_D + \|\mathbf{M}\mathbf{v}_\pi - \mathbf{v}_\pi\|_D$$

整理得：

$$\|\Phi \mathbf{w}^* - \mathbf{v}_\pi\|_D \leq \frac{1}{1-\gamma} \|\mathbf{M} \mathbf{v}_\pi - \mathbf{v}_\pi\|_D = \frac{1}{1-\gamma} \min_{\mathbf{w}} \|\hat{\mathbf{v}}(\mathbf{w}) - \mathbf{v}_\pi\|_D$$

其中最后一个等式是因为 $\|\mathbf{M} \mathbf{v}_\pi - \mathbf{v}_\pi\|_D$ 是 $\mathbf{v}_\pi$ 与其在所有可能近似空间上的正交投影之间的误差，因此是最小可能误差。

以下证明已在上文中使用的事实：

- $\|\mathbf{M}\|_D = 1$ ：因为 $\|\mathbf{M}\|_D = \|\mathbf{D}^{1/2} \Phi (\Phi^T \mathbf{D} \Phi)^{-1} \Phi^T \mathbf{D} \mathbf{D}^{-1/2}\|_2 = 1$ ，其中 L_2 -范数下的矩阵是正交投影矩阵，其 L_2 -范数恒为 1。
- $\|\mathbf{P}_\pi \mathbf{x}\|_D \leq \|\mathbf{x}\|_D$ ：由 $\|\mathbf{P}_\pi \mathbf{x}\|_D^2 = \mathbf{x}^T \mathbf{P}_\pi^T \mathbf{D} \mathbf{P}_\pi \mathbf{x} = \sum_k [\mathbf{D}]_{kk} (\sum_i [\mathbf{P}_\pi]_{ki} x_i)^2 \leq \sum_k [\mathbf{D}]_{kk} (\sum_i [\mathbf{P}_\pi]_{ki} x_i^2) = \sum_i (\sum_k [\mathbf{D}]_{kk} [\mathbf{P}_\pi]_{ki}) x_i^2 = \sum_i [\mathbf{D}]_{ii} x_i^2 = \|\mathbf{x}\|_D^2$ ，其中利用了 Jensen 不等式和 $\mathbf{d}_\pi^T \mathbf{P}_\pi = \mathbf{d}_\pi^T$ 。

最小二乘 TD (LSTD)

接下来介绍一种称为**最小二乘 TD** (Least-Squares TD, LSTD) 的算法。与 TD-Linear 算法一样，LSTD 也旨在最小化投影贝尔曼误差，但具有一些优势。

回顾最小化投影贝尔曼误差的最优参数为 $\mathbf{w}^* = \mathbf{A}^{-1} \mathbf{b}$ 。事实上，由 (8.27) 可知 $\mathbf{A}$ 和 $\mathbf{b}$ 也可以写作期望形式：

$$\mathbf{A} = \mathbb{E}[\phi(s_t)(\phi(s_t) - \gamma \phi(s_{t+1}))^T], \quad \mathbf{b} = \mathbb{E}[r_{t+1} \phi(s_t)]$$

LSTD 的思想很简单：如果能用随机样本直接获得 $\mathbf{A}$ 和 $\mathbf{b}$ 的估计 $\hat{\mathbf{A}}$ 和 $\hat{\mathbf{b}}$ ，则最优参数可直接估计为 $\mathbf{w}^* \approx \hat{\mathbf{A}}^{-1} \hat{\mathbf{b}}$ 。

具体而言，假设 $(s_0, r_1, s_1, \dots, s_t, r_{t+1}, s_{t+1}, \dots)$ 是遵循给定策略 π 的轨迹。设 $\hat{\mathbf{A}}_t$ 和 $\hat{\mathbf{b}}_t$ 分别为时刻 t 时 $\mathbf{A}$ 和 $\mathbf{b}$ 的估计值，作为样本的平均值：

$$\hat{\mathbf{A}}_t = \sum_{k=0}^{t-1} \phi(s_k)(\phi(s_k) - \gamma \phi(s_{k+1}))^T, \quad \hat{\mathbf{b}}_t = \sum_{k=0}^{t-1} r_{k+1} \phi(s_k)$$

则估计参数为 $\mathbf{w}_t = \hat{\mathbf{A}}_t^{-1} \hat{\mathbf{b}}_t$ 。由于 $\hat{\mathbf{A}}_t$ 在 t 较小时可能不可逆，通常将其偏移一个小常数矩阵 $\varepsilon \mathbf{I}$ ($\varepsilon > 0$)。

LSTD 的优点是比 TD 方法更高效地利用经验样本，收敛更快。其缺点包括：只能估计状态价值（而 TD 算法可以扩展为估计动作价值）；只允许线性近似器；计算成本更高（每步需更新 $m \times m$ 矩阵并计算其逆，复杂度 $O(m^3)$ ）。

解决矩阵求逆问题的方法是利用递归最小二乘，直接更新 $\hat{\mathbf{A}}_t$ 的逆而非 $\hat{\mathbf{A}}_t$ 本身：

$$\begin{aligned} \hat{\mathbf{A}}_{t+1} &= \sum_{k=0}^t \phi(s_k)(\phi(s_k) - \gamma \phi(s_{k+1}))^T \\ &= \hat{\mathbf{A}}_t + \phi(s_t)(\phi(s_t) - \gamma \phi(s_{t+1}))^T \end{aligned}$$

其逆可通过 Sherman-Morrison 公式递归计算：

$$\hat{\mathbf{A}}_{t+1}^{-1} = \hat{\mathbf{A}}_t^{-1} - \frac{\hat{\mathbf{A}}_t^{-1} \phi(s_t)(\phi(s_t) - \gamma \phi(s_{t+1}))^T \hat{\mathbf{A}}_t^{-1}}{1 + (\phi(s_t) - \gamma \phi(s_{t+1}))^T \hat{\mathbf{A}}_t^{-1} \phi(s_t)}$$

该递归算法不需要步长，但需要设置 $\hat{\mathbf{A}}_0^{-1}$ 的初始值，通常选择 $\hat{\mathbf{A}}_0^{-1} = \varepsilon \mathbf{I}$ ($\varepsilon > 0$)。

8.3 基于函数近似的动作价值 TD 学习

8.2 节介绍了状态价值估计问题，本节介绍如何估计**动作价值**。将表格 Sarsa 和表格 Q-learning 算法扩展到价值函数近似的情形，扩展非常直接。

8.3.1 带函数近似的 Sarsa

带函数近似的 Sarsa 算法可直接从 (8.13) 将状态价值替换为动作价值得到。假设 $q_\pi(s, a)$ 被近似为 $\hat{q}(s, a; \mathbf{w})$ 。将 (8.13) 中的 $\hat{v}(s; \mathbf{w})$ 替换为 $\hat{q}(s, a; \mathbf{w})$ 得：

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}; \mathbf{w}_t) - \hat{q}(s_t, a_t; \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{q}(s_t, a_t; \mathbf{w}_t) \quad (15)$$

其分析与 (8.13) 类似，此处从略。当使用线性函数时有：

$$\hat{q}(s, a; \mathbf{w}) = \phi^T(s, a) \mathbf{w}$$

其中 $\phi(s, a)$ 是特征向量。此时 $\nabla_{\mathbf{w}} \hat{q}(s, a; \mathbf{w}) = \phi(s, a)$ 。

(8.35) 中的价值估计步骤可以与策略改进步骤结合以学习最优策略。该过程总结在算法 8.2 中。需要注意的是，准确估计给定策略的动作价值要求 (8.35) 运行充分多次，但在算法 8.2 中 (8.35) 仅在策略改进步骤前执行一次，这与表格 Sarsa 算法类似。此外，算法 8.2 的目的是找到一条从指定起始状态到目标状态的良好路径，因此无法找到所有状态的最优策略。然而，如果有足够的经验数据，可以很容易地修改实现过程以找到所有状态的最优策略。

备注

算法 8.2：带函数近似的 Sarsa

初始化：初始参数 $\mathbf{w}_0$ 。初始策略 π_0 。 $\alpha_t = \alpha > 0$ 对所有 t 。 $\varepsilon \in (0, 1)$ 。

目标：学习一个最优策略，使智能体从初始状态 s_0 到达目标状态。

对每个 episode：

1. 在 s_0 处按 $\pi_0(s_0)$ 生成 a_0
2. 若 s_t ($t = 0, 1, 2, \dots$) 不是目标状态，则执行：
 - 在 (s_t, a_t) 下收集经验样本 $(r_{t+1}, s_{t+1}, a_{t+1})$ ：通过与环境的交互生成 r_{t+1}, s_{t+1} ；按 $\pi_t(s_{t+1})$ 生成 a_{t+1} 。
 - 更新 q 值：

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t [r_{t+1} + \gamma \hat{q}(s_{t+1}, a_{t+1}; \mathbf{w}_t) - \hat{q}(s_t, a_t; \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{q}(s_t, a_t; \mathbf{w}_t)$$

- 更新策略 (ε -greedy)：

$$\pi_{t+1}(a|s_t) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a; \mathbf{w}_{t+1}) \\ \frac{\varepsilon}{|\mathcal{A}(s_t)|}, & \text{否则} \end{cases}$$

- $s_t \leftarrow s_{t+1}, a_t \leftarrow a_{t+1}$

8.3.2 带函数近似的 Q-learning

表格 Q-learning 也可以扩展到函数近似的情形。更新规则为：

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t [r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a; \mathbf{w}_t) - \hat{q}(s_t, a_t; \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{q}(s_t, a_t; \mathbf{w}_t) \quad (16)$$

上述更新规则与 (8.35) 类似，只是将 (8.35) 中的 $\hat{q}(s_{t+1}, a_{t+1}; \mathbf{w}_t)$ 替换为 $\max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a; \mathbf{w}_t)$ 。

与表格情形类似，(8.36) 既可以实现为 on-policy 形式也可以实现为 off-policy 形式。on-policy 版本见算法 8.3。

备注

算法 8.3: 带函数近似的 Q-learning (on-policy 版本)

初始化: 初始参数 $\mathbf{w}_0$ 。初始策略 π_0 。 $\alpha_t = \alpha > 0$ 对所有 t 。 $\varepsilon \in (0, 1)$ 。

目标: 学习一条从初始状态 s_0 到目标状态的最优路径。

对每个 episode:

1. 若 s_t ($t = 0, 1, 2, \dots$) 不是目标状态, 则执行:

- 在 s_t 下收集经验样本 (a_t, r_{t+1}, s_{t+1}) : 按 $\pi_t(s_t)$ 生成 a_t ; 通过与环境的交互生成 r_{t+1}, s_{t+1} 。
- 更新 q 值:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t [r_{t+1} + \gamma \max_{a \in \mathcal{A}(s_{t+1})} \hat{q}(s_{t+1}, a; \mathbf{w}_t) - \hat{q}(s_t, a_t; \mathbf{w}_t)] \nabla_{\mathbf{w}} \hat{q}(s_t, a_t; \mathbf{w}_t)$$

- 更新策略 (ε -greedy):

$$\pi_{t+1}(a|s_t) = \begin{cases} 1 - \frac{\varepsilon}{|\mathcal{A}(s_t)|} (|\mathcal{A}(s_t)| - 1), & a = \arg \max_{a \in \mathcal{A}(s_t)} \hat{q}(s_t, a; \mathbf{w}_{t+1}) \\ \frac{\varepsilon}{|\mathcal{A}(s_t)|}, & \text{否则} \end{cases}$$

带线性函数近似的 Q-learning 可以成功学习最优策略, 这里使用五阶线性傅里叶基函数。off-policy 版本将推迟到 8.4 节介绍深度 Q-learning 时演示。

值得注意的是, 在算法 8.2 和算法 8.3 中, 虽然价值是用函数表示的, 但策略 $\pi(a|s)$ 仍然以表格形式表示, 因此仍需假设状态和动作数量有限。第 9 章将展示策略也可以用函数表示, 从而处理连续状态和动作空间。

8.4 深度 Q-learning

我们可以将深度神经网络融入 Q-learning, 得到深度 Q-learning 或深度 Q 网络 (Deep Q-Network, DQN)。深度 Q-learning 是最早也是最成功的深度强化学习算法之一。值得注意的是, 神经网络不必很深, 对于本书的网格世界等简单任务, 一到两个隐藏层的浅层网络可能就足够了。

深度 Q-learning 可以看作算法 (8.36) 的扩展。但其数学形式化和实现技术与之前有本质不同, 值得特别关注。

8.4.1 算法描述

从数学角度来看, 深度 Q-learning 旨在最小化以下目标函数:

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w}) - \hat{q}(S, A; \mathbf{w}) \right)^2 \right] \quad (17)$$

其中 (S, A, R, S') 是分别表示状态、动作、即时奖励和下一状态的随机变量。该目标函数可以看作贝尔曼最优误差的平方。这是因为

$$q(s, a) = \mathbb{E} \left[R_{t+1} + \gamma \max_{a \in \mathcal{A}(S_{t+1})} q(S_{t+1}, a) \mid S_t = s, A_t = a \right], \quad \forall s, a$$

是贝尔曼最优方程。因此, 当 $\hat{q}(S, A; \mathbf{w})$ 能精确近似最优动作价值时, $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w}) - \hat{q}(S, A; \mathbf{w})$ 在期望意义下应等于零。

为最小化目标函数 (8.37), 可使用梯度下降算法。需要计算 J 关于 $\mathbf{w}$ 的梯度。注意参数 $\mathbf{w}$ 不仅出现在 $\hat{q}(S, A; \mathbf{w})$ 中, 还出现在 $y := R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w})$ 中, 因此直接计算梯度并非易事。为简化计算, 假设 y 中的 $\mathbf{w}$ (在短时间内) 固定不变, 从而使得梯度计算变得容易得多。

具体而言，引入两个网络：一个是**主网络**，表示 $\hat{q}(s, a; \mathbf{w})$ ；另一个是**目标网络** $\hat{q}(s, a; \mathbf{w}_T)$ 。此时目标函数变为：

$$J = \mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w}_T) - \hat{q}(S, A; \mathbf{w}) \right)^2 \right] \quad (18)$$

其中 $\mathbf{w}_T$ 是目标网络的参数。当 $\mathbf{w}_T$ 固定时， J 的梯度为：

$$\nabla_{\mathbf{w}} J = -\mathbb{E} \left[\left(R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w}_T) - \hat{q}(S, A; \mathbf{w}) \right) \nabla_{\mathbf{w}} \hat{q}(S, A; \mathbf{w}) \right] \quad (19)$$

要利用 (8.38) 中的梯度来最小化目标函数，需要注意以下关键技术。

- **双网络技术**：如上所述，使用一个主网络和一个目标网络。设 $\mathbf{w}$ 和 $\mathbf{w}_T$ 分别为主网络和目标网络的参数，初始时设为相同。

每次迭代中，从**经验回放缓冲区**中抽取一个小批量的样本 $\{(s, a, r, s')\}$ 。主网络的输入是 s 和 a ，输出 $y = \hat{q}(s, a; \mathbf{w})$ 为估计的 q 值。输出的目标值为 $y_T := r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a; \mathbf{w}_T)$ 。通过最小化小批量样本上的 TD 误差（也称为损失函数） $\sum (y - y_T)^2$ 来更新主网络。

主网络在每次迭代中更新。目标网络则每经过一定次数的迭代后设置为主网络的副本，以满足计算梯度时 $\mathbf{w}_T$ 固定的假设。

- **经验回放**：收集到一些经验样本后，不是按收集顺序使用它们，而是将它们存储在一个称为**经验回放缓冲区**的数据集中。设 (s, a, r, s') 为一个经验样本， $\mathcal{B} := \{(s, a, r, s')\}$ 为回放缓冲区。每次更新主网络时，从回放缓冲区中按**均匀分布**抽取一个小批量经验样本。

为什么深度 Q-learning 需要经验回放，又为什么必须遵循均匀分布？答案在于目标函数 (8.37)。为良好定义目标函数，必须确定 S, A, R, S' 的概率分布。一旦给定了 (S, A) ， R 和 S' 的分布由系统模型决定。描述状态-动作对 (S, A) 分布的最简单方式是假设其均匀分布。

然而，在实践中状态-动作样本是序列化生成的，不一定均匀分布，因此需要打破序列中样本之间的相关性来满足均匀分布的假设。通过从回放缓冲区中均匀抽样来实现经验回放，正是出于这一数学原因。随机采样的一个额外好处是每个经验样本可能被多次使用，从而提高了数据效率。

深度 Q-learning 的实现过程总结在算法 8.4 中。该实现是 off-policy 的，如有需要也可以改为 on-policy。

备注

算法 8.4：深度 Q-learning (off-policy 版本)

初始化：一个主网络和一个目标网络，具有相同的初始参数。

目标：学习一个最优目标网络，从给定行为策略 π_b 生成的经验样本中近似最优动作价值。

1. 将 π_b 生成的经验样本存储到回放缓冲区 $\mathcal{B} = \{(s, a, r, s')\}$ 中

2. 每次迭代：

- 从 $\mathcal{B}$ 中均匀抽取一个小批量样本
- 对每个样本 (s, a, r, s') ，计算目标值 $y_T = r + \gamma \max_{a \in \mathcal{A}(s')} \hat{q}(s', a; \mathbf{w}_T)$ ，其中 $\mathbf{w}_T$ 是目标网络的参数
- 利用小批量样本更新主网络，最小化 $(y_T - \hat{q}(s, a; \mathbf{w}))^2$
- 每隔 C 次迭代设置 $\mathbf{w}_T = \mathbf{w}$

8.4.2 示例说明

以下示例演示算法 8.4，目标是学习所有状态-动作对的最优动作价值。

一个 episode 由图 8.11(a) 所示的行为策略生成。该行为策略是探索性的：在任何状态下选择任何动作的概率相同。如图所示，episode 仅有 1000 步。由于行为策略的强探索能力，episode 中几乎

所有的状态-动作对都被访问了。回放缓冲区包含 1000 个经验样本。小批量大小为 100，意味着每次从回放缓冲区中均匀抽取 100 个样本。

主网络和目标网络具有相同的结构：一个带有 100 个神经元隐藏层的神经网络。该神经网络有三个输入和一个输出。前两个输入是状态的归一化行索引和列索引，第三个输入是归一化后的动作索引（“归一化”指将值映射到 $[0, 1]$ 区间）。网络输出是估计价值。之所以将输入设计为状态的行列坐标而非状态索引，是因为已知状态对应网格中的二维位置，使用的状态信息越多，网络性能越好。此外，神经网络也可以有其他设计方式，例如有两个输入和五个输出（两个输入为归一化的行列坐标，输出为输入状态下五个动作的估计价值）。

损失函数（定义为每个小批量中平均平方 TD 误差）收敛到零，意味着网络可以很好地拟合训练样本。状态价值估计误差也收敛到零，表明最优动作价值的估计足够精确。因此，对应的贪心策略是最优的。

该示例展示了深度 Q-learning 的高效性。特别是仅需 1000 步的短 episode 就足以获得最优策略，而表格 Q-learning 需要 100,000 步。高效的原因之一是函数近似方法具有较强的泛化能力，另一个原因是经验样本可以被重复使用。

我们接着有意用更少的经验样本来挑战深度 Q-learning 算法。图 8.12 展示了一个仅有 100 步的 episode 示例。此时，虽然损失函数仍然可以收敛到零（网络可以很好地拟合给定的经验样本），但状态估计误差无法收敛到零。这意味着经验样本太少，不足以精确估计最优动作价值。

8.5 本章小结

本章继续介绍 TD 学习算法，但核心从表格方法切换到了函数近似方法。理解函数近似方法的关键在于认识到它是一个优化问题。最简单的目标函数是真实状态价值与估计值之间的平方误差，此外还有贝尔曼误差和投影贝尔曼误差等目标函数。本章证明了 TD-Linear 算法实际上是在最小化投影贝尔曼误差。还介绍了 Sarsa 和 Q-learning 等优化算法在价值近似下的扩展。

价值函数近似方法之所以重要，原因之一是它使人工神经网络能够与强化学习相结合。例如，深度 Q-learning 就是最成功的深度强化学习算法之一。虽然神经网络作为非线性函数近似器已被广泛使用，但本章对线性函数情形进行了全面的介绍，充分理解线性情形对于更好地理解非线性情形至关重要。

平稳分布是本章引入的一个重要概念。它在价值函数近似方法中定义合适的目标函数时发挥着关键作用，也将在第 9 章中使用函数近似策略时发挥关键作用。

8.6 常见问题解答

• Q：表格方法和函数近似方法有什么区别？

A：一个重要区别是价值的更新和检索方式。

如何检索价值：表格方法中可以直接读取表中对应的条目。函数近似方法则需要将状态 s 输入函数并计算函数值。如果函数是人工神经网络，还需要进行从前向传播的过程。

如何更新价值：表格方法中可以直接重写表中对应的条目。函数近似方法则必须通过更新函数参数来间接改变价值。

• Q：函数近似方法相比表格方法有哪些优点？

A：由于状态价值的检索方式不同，函数近似方法在存储上更高效。表格方法需要存储 $|S|$ 个值，而函数近似方法只需要存储一个参数向量，其维数通常远小于 $|S|$ 。由于状态价值的更新方式不同，函数近似方法还具有更强的泛化能力：更新函数参数会同时影响多个状态的价值，因此一个状态的经验样本可以泛化帮助估计其他状态的价值。

• Q：表格方法和函数近似方法可以统一吗？

A：可以。表格方法可以视为函数近似方法的一种特殊情形。详细内容见专栏 8.2。

- **Q: 什么是平稳分布？为什么它很重要？**

A: 平稳分布描述马尔可夫决策过程的长期行为。智能体在充分长时间内执行某个策略后，访问各状态的概率可由平稳分布描述（详见专栏 8.1）。

平稳分布之所以在本章出现，是因为定义有效的目标函数需要它。目标函数涉及状态的概率分布，通常选择平稳分布。平稳分布不仅对价值近似方法重要，对第 9 章将介绍策略梯度方法也很关键。

- **Q: 线性函数近似方法有哪些优点和缺点？**

A: 线性函数近似是最简单的情形，其理论性质可以得到全面分析。但它的近似能力有限，对于复杂任务也难以选择合适的特征向量。相比之下，人工神经网络可以作为黑盒通用非线性近似器来近似价值，使用更加友好。然而，研究线性情形仍然有助于更好地掌握函数近似方法的思想。此外，线性情形能力也很强大，因为表格方法可以视为一种特殊的线性情形（专栏 8.2）。

- **Q: 为什么深度 Q-learning 需要经验回放？**

A: 原因在于目标函数 (8.37)。为良好定义目标函数，必须确定 S, A, R, S' 的概率分布。一旦 (S, A) 给定， R 和 S' 的分布由系统模型决定。描述状态-动作对 (S, A) 分布的最简单方式是假设其均匀分布。然而实践中状态-动作样本是序列化生成的，不一定均匀分布，因此需要打破序列中样本的相关性来满足均匀分布的假设。通过从回放缓冲区中均匀抽样来进行经验回放，正是出于这一数学原因。经验回放的一个额外好处是每个样本可能被多次使用，提高数据效率。

- **Q: 表格 Q-learning 可以使用经验回放吗？**

A: 虽然表格 Q-learning 不要求经验回放，但由于其 off-policy 属性，它对样本的获取方式没有限制，因此也可以使用经验回放而不会遇到问题。好处是样本可以被重复使用，更加高效。

- **Q: 为什么深度 Q-learning 需要两个网络？**

A: 根本原因是为了简化目标函数 (8.37) 梯度的计算。由于 $\mathbf{w}$ 不仅出现在 $\hat{q}(S, A; \mathbf{w})$ 中，还出现在 $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w})$ 中，直接计算梯度并非易事。一方面，如果固定 $R + \gamma \max_{a \in \mathcal{A}(S')} \hat{q}(S', a; \mathbf{w})$ 中的 $\mathbf{w}$ ，梯度可以容易地计算，如 (8.38) 所示。这暗示应当维持两个网络：主网络的参数每次迭代更新，目标网络的参数在特定周期内保持不变。另一方面，目标网络的参数不能永远固定，应当每隔一定迭代次数进行更新。

- **Q: 当使用人工神经网络作为非线性函数近似器时，如何更新其参数？**

A: 必须注意不应直接使用 (8.36) 等方式更新参数向量，而应遵循神经网络的训练流程来更新参数。这一流程可以借助目前已成熟且广泛可用的神经网络训练工具包来实现。